1. Validación de documentos XML



En la Unidad 1 se dispusieron las bases del funcionamiento de XML. En ellas se indicó lo que se consideraba un documento XML **bien formado**. Los documentos bien formados, aseguran que las reglas de XML se cumplen y que no hay ninguna incoherencia al usar el lenguaje.

Sin embargo, no es suficiente porque podríamos definir documentos que utilizaran los elementos que quisiéramos sin restricción.

En la realidad los elementos y los atributos que se pueden utilizar y la manera de disponerles en el documento es fundamental para mantener una mayor homogeneidad.

Una empresa puede decidir que los documentos internos para describir el software que utiliza la empresa deben poseer como elemento raíz un elemento llamado **software** (y no otro) y que este elemento obligatoriamente debe contener los elementos **nombre**, **fabricante** y **precio**.

Esas reglas no se refieren a que el documento esté bien formado; son reglas más complejas y que permitirán al documento que sea válido.

Para ello se crea un documento que contendrá las reglas que deben de cumplir los XML que se basen en él. De modo que un documento deberá indicar qué plantilla de reglas utiliza y deberá cumplirlas a rajatabla para considerarse válido.

Así explicado parece que la validación supone un problema, pero en realidad es una ventaja; con la validación tenemos la seguridad de que los documentos cumplen unas reglas más concretas y de esa forma es fácil establecer un protocolo en las empresas para sus documentos.

De hecho cuando un documento XML cumple estrictamente las reglas generales de creación XML se dice que **está bien formado** (documento sintácticamente correcto).

Cuando además sigue las reglas de un documento de validación (o documento patrón) entonces se dice que es **válido**.

Las técnicas más populares para validar documentos son:

DTD, **Document Type Definition**. Validación por documentos de definición de tipos. Se utilizaba en el lenguaje SGML y de ahí debe su popularidad. Es la más utilizada, pero tiene numerosas voces críticas porque su sintaxis no es XML.

XML Schema o esquemas XML. Mucho más coherente con el lenguaje XML es la aconsejada actualmente.

2. Validación por DTD

Es la técnica más veterana y, en realidad procede de **SGML** el lenguaje base de XML. Es indudablemente la más utilizada, pero también la menos coherente con las reglas XML. Su éxito se debe a que ya era una forma de validación reconocida antes de la aparición de XML, por lo que muchísimo productos software la reconocen desde hace mucho y la compatibilidad con ellos ha determinado su éxito.

3. Posibilidades de uso de DTD

1ª de 3, EN EL PROPIO DOCUMENTO XML

Se puede definir la estructura que debe cumplir un documento XML mediante código DTD insertado en el propio documento. La desventaja evidente, es que esta definición sólo vale para dicho documento, por lo que realmente no define **tipos** de documentos XML (no es una plantilla en definitiva de definición de documentos). Por ello es la forma menos habitual de utilizar DTD.

Su única (pero muy discutible) ventaja es que la validación está dentro del propio documento, por lo que siempre viajan juntos la validación y el contenido del mismo.

Un documento XML que defina internamente su DTD, simplemente escribe instrucciones DTD dentro del propio documento dentro de una etiqueta **DOCTYPE**.

SINTAXIS:

```
<!DOCTYPE raíz [....códigoDTD...]>
```

EJEMPLO:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
        <!ELEMENT persona (nombre)>
        <!ELEMENT nombre (#PCDATA)>
]>
<persona>
        <nombre>Antonio</nombre>
</persona>
```



Prueba el código anterior en el editor y comprueba que además de bien formado, se trata de un documento válido.

2ª de 3, EN UN DOCUMENTO EXTERNO PRIVADO

En este caso la validación se crea en un documento-plantilla externa. De modo que cuando un documento deba cumplir las reglas de la plantilla DTD, se debe indicar la ruta (relativa o absoluta) a la misma.

SINTAXIS CON RUTA ABSOLUTA:

```
<!DOCTYPE raíz SYSTEM "http://www.empresa.com/docs.dtd">
```

SINTAXIS CON RUTA RELATIVA:

```
<!DOCTYPE raíz SYSTEM "documento.dtd">
```

Observa el siguiente ejemplo donde se observa la vinculación de un xml con su dtd.

agenda.xml

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>

<!DOCTYPE agenda SYSTEM "dtdagenda.dtd">

<agenda>
    <persona >
        <nombre>José España</nombre>
        <tlf>951345678 </tlf>
    </persona>

    <persona>
        <nombre>Eva Pérez</nombre>
        <tlf>955837659</tlf>
    </persona>

    <persona >
        <nombre>Marta Sánchez</nombre>
        <tlf>955837659</tlf>
    </persona>

 </agenda>
```

dtdagenda.dtd

```
<!ELEMENT agenda (persona)+>
<!ELEMENT persona (nombre, tlf)>
<!ELEMENT nombre (#PCDATA)>
<!ELEMENT tlf (#PCDATA)>
```



Prueba en el editor que estamos utilizando que **agenda.xml** se trata de un documento **válido** según el patrón definido en **dtdagenda.dtd**.

3ª de 3, EN UN DOCUMENTO EXTERNO PUBLICO

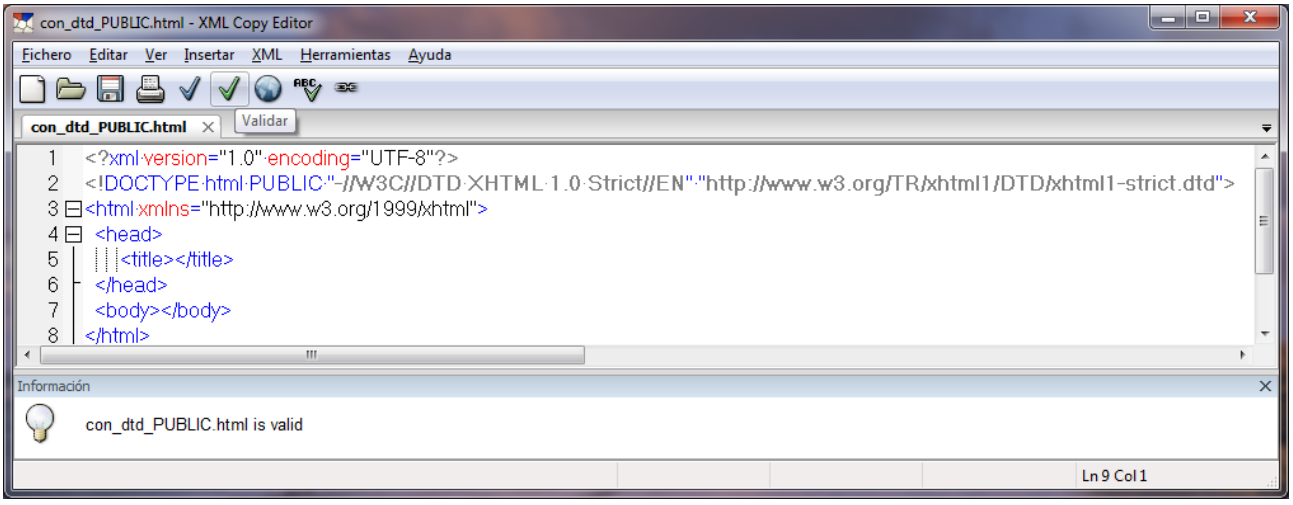
Se entiende que **SYSTEM** se utiliza cuando el documento DTD es privado. Si se trata de un documento de uso público, entonces se usa **PUBLIC**. La sintaxis sería:

SINTAXIS

<!DOCTYPE raíz **PUBLIC** "nombreDTD" "DTD_URL"**>**

La **raíz** sigue siendo el nombre del elemento raíz. El **nombreDTD** es el nombre público que se le da al DTD en cuestión.

Observa la siguiente imagen, que puedes generar con el editor con el que estamos trabajando:



Este es el DOCTYPE para una página web escrita en XHTML 1.0 estricto, utilizada para validar miles de páginas web.

ATRIBUTO standalone DE LA ETIQUETA CABECERA (PRÓLOGO)

La etiqueta **<?xml** de cabecera de todo documento XML, posee un atributo llamado **standalone** que puede tomar dos valores:

yes. En caso de el documento XML no utilice DTD externa (es independiente).

no. Cuando el documento obligatoriamente hace uso de DTD externa.

Para apreciar esto último, observa la asociación xml-dtd anterior. Mira el prólogo del xml y el valor del atributo **standalone**.

4. Definiciones en un DTD



En un código DTD (tanto externo como interno) se pueden definir:

- Los **elementos** que se pueden utilizar en un documento XML. En esta definición se indica además qué pueden contener dichos elementos.
- Los **atributos** que pueden poseer los elementos. Además incluso indicando sus posibles valores válidos.
- **Entidades** que puede utilizar el documento XML.

5. Elementos



Mediante un DTD podemos especificar el elemento que se puede utilizar en un XML se define en su DTD mediante una etiqueta **!ELEMENT**. La sintaxis de la misma es:

```
<!ELEMENT nombre tipo>
```

El **nombre** es el identificador que tendrá el elemento en el documento XML (hay que recordar que se distingue entre mayúsculas y minúsculas).

El **tipo** indica el funcionamiento del elemento, relativo al contenido que puede tener. A continuación se indican las posibilidades de este parámetro

TIPOS DE CONTENIDO EN LOS ELEMENTOS

EMPTY

Significa que el elemento no podrá tener contenido alguno, es un elemento vacío (como la etiqueta **br** de las páginas web). Ejemplo de definición de elemento vacío:

```
<!ELEMENT línea EMPTY >
```

Cuando se indica como tipo la palabra EMPTY, se indica que el elemento no puede tener contenido, pero sí podrá contener atributos (si se especifican en el DTD).

Los elementos definidos con EMPTY sólo pueden utilizarse de esta forma:

```
<alumno></alumno>     o bien     <alumno/>
```

EJEMPLO:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE nombre [
        <!ELEMENT nombre EMPTY>
]>

<nombre></nombre>
```

En el editor, introduce algún valor en la etiqueta “nombre”. Comprueba que no es válido.

ANY

Permite cualquier contenido en el elemento, sin restricciones de ningún tipo. Es decir puede contener texto, otro tipo de datos y cualquier etiqueta. Además puede tener atributos.

EJEMPLO:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
        <!ELEMENT persona (nombre, apellidos)>
        <!ELEMENT nombre (#PCDATA)>
        <!ELEMENT apellidos ANY>
]>
<persona>
        <nombre>Jorge</nombre>
        <apellidos>Sánchez Asenjo
                <nombre>Jorge</nombre>
        </apellidos>
</persona>
```

Al definir **apellidos** como elemento ANY, permite incluso que dentro haya una etiqueta **nombre**.

Puesto que un DTD se usa para restringir la escritura de un tipo de documentos XML, el uso de ANY debe de ser muy cauteloso.

ELEMENTO CONCRETO

En los elementos se puede indicar claramente un contenido concreto para el mismo. Dicho contenido se indica entre paréntesis.

EJEMPLO:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
<!ELEMENT persona (nombre)>
<!ELEMENT nombre (#PCDATA)>
]>
<persona>
        <nombre>Antonio</nombre>
</persona>
```

En el ejemplo dentro de una etiqueta **persona obligatoriamente** debe de existir una etiqueta **nombre** (**una y sólo una**).

#PCDATA significa que el elemento podrá contener texto (tan largo como se desee).

SECUENCIAS

En el caso de indicar una lista de elementos separados por comas, indica que el elemento contendrá la lista de elementos indicada, la cual deberá estar en el mismo orden de la secuencia. Es decir en el ejemplo los **apellidos** no se podrían poner delante del **nombre**.

EJEMPLO:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE persona [
        <!ELEMENT persona (nombre, apellidos, edad)>
        <!ELEMENT nombre (#PCDATA)>
        <!ELEMENT apellidos (#PCDATA)>
        <!ELEMENT edad (#PCDATA)>
]>
<persona>
        <nombre>Antonio</nombre>
        <apellidos>Pérez</apellidos>
        <edad>35</edad>
</persona>
```

ELECCIONES

Los elementos pueden contener elementos opcionales (puede aparecer uno u otro).

EJEMPLO:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE artículo [
        <!ELEMENT artículo (id | serie)>
        <!ELEMENT id (#PCDATA)>
        <!ELEMENT serie (#PCDATA)>
]>
<artículo>
        <id>16</id>
</artículo>
```

La barra vertical indica que el elemento puede contener una u otra opción (pero sólo una). Es decir también sería válido:

EJEMPLO:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE artículo [
        <!ELEMENT artículo (id | serie)>
        <!ELEMENT id (#PCDATA)>
        <!ELEMENT serie (#PCDATA)>
]>
<artículo>
        <serie>abc2258-z</serie>
</artículo>
```

COMBINACIONES

Por supuesto puede haber combinaciones, si tenemos un documento DTD llamado **coordenada.dtd** con este contenido:

Documento: coordenada.dtd

```
<!ELEMENT coordenada ((longitud, latitud) | HusoUTM)>
<!ELEMENT longitud (#PCDATA)>
<!ELEMENT latitud (#PCDATA)>
<!ELEMENT HusoUTM (#PCDATA)>
```

Este documento sería válido:

Documento: XML

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE coordenada SYSTEM "coordenada.dtd">
<coordenada>
        <longitud>5</longitud>
        <latitud>39</latitud>
</coordenada>
```

Y este otro, también:

Documento: XML

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE coordenada SYSTEM "coordenada.dtd">
<coordenada>
        <HusoUTM>30</HusoUTM>
</coordenada>
```

CARDINALIDAD

La cardinalidad es el número de veces que puede aparecer un determinado contenido en un elemento. Se realiza mediante estos símbolos:

- **?** Contenido **opcional**, puede no aparecer, o aparecer una sola vez (0,1).
- ***** Contenido **opcional** y **repetible**. Puede no aparecer y puede, incluso, aparecer varias veces (0,n).
- **+** Contenido **obligatorio** y **repetible**. Tiene que aparecer e incluso puede aparecer varias veces (1,n).

Ejemplo:

```
<!ELEMENT película (título, dirección+, argumento?, actor*)>
```



Codifica un documento XML que pueda validarse con un DTD, cuya primera línea sea como la que aparece en el código anterior.

Otro ejemplo. Partiremos del documento **polígono.dtd** que validará los dos códigos **XML** que también se muestran a continuación.

Documento: poligono.dtd

```
<!ELEMENT polígono ((coordX,coordY)+ | nombre)>
<!ELEMENT coordX (#PCDATA)>
<!ELEMENT coordY (#PCDATA)>
<!ELEMENT nombre (#PCDATA)>
```

Este documento sería válido:

Documento: XML

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE polígono SYSTEM "poligono.dtd">
<polígono>
        <coordX>12</coordX>
        <coordY>13</coordY>
        <coordX>17</coordX>
        <coordY>23</coordY>
        <coordX>34</coordX>
        <coordY>56</coordY>
</polígono>
```

Y este otro, también:

Documento: XML

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE polígono SYSTEM "poligono.dtd">
<polígono>
        <nombre>triángulo</nombre>
</polígono>
```

6. Declaración de atributos



Los atributos permiten añadir información a un elemento. Un atributo no puede constar de más atributos y cada atributo sólo puede aparecer una vez en cada elemento.

SINTAXIS DE LA DECLARACIÓN DE ATRIBUTOS

Los atributos se declaran mediante la etiqueta !ATTLIST, sintaxis:

```
<!ATTLIST elemento nombreAtributo tipo presencia valorPorDefecto>
```

Donde:

elemento. Es el nombre del elemento que podrá utilizar el atributo.

nombreAtributo. Es el identificador del atributo que estamos declarando.

tipo. Es el tipo de valores que podemos asignar al atributo.

presencia. Indica las características de los valores

valorPorDefecto. Permite dar un valor que el atributo tomará en el documento XML en caso de que no se le dé en el mismo ningún valor al atributo. También indica si es necesario rellenar o no el atributo o bien si es opcional.

CDATA

Para indicar el tipo de valores de un atributo se usa la palabra **CDATA** para indicar que el atributos contiene texto (CDATA es el acrónimo de **Character DATA**).

A diferencia de **#PCDATA**, su contenido no es procesado, lo que significa que puede contener cualquier valor (incluidos símbolos prohibidos en los #PCDATA como **<**, **>**, **&**)

Declaración de atributos

Esta declaración:

DTD:

```
<!ATTLIST persona nacionalidad CDATA>
```

Significa que hemos definido el atributo **nacionalidad** correspondiente al elemento **persona**. Que será de tipo **CDATA**, es decir texto normal. Así en un XML que se valide con el DTD en el que está la instrucción anterior, podremos:

XML:

```
<persona nacionalidad=”española”>
```

Valores por defecto

Al declarar un atributo, lo último que se indica es la propiedad relativa al valor por defecto del atributo. Se comentan a continuación sus posibilidades.

Valor por defecto concreto

Si al final de la declaración de un atributo aparece un valor concreto, se entiende que ese será el valor por defecto. Es decir que se podría no utilizar el atributo en un elemento y entonces dicho atributo tomaría dicho valor.

Analicemos los siguientes códigos:

DTD: directorio.dtd

```
<!ELEMENT directorio (persona)+>
<!ELEMENT persona (#PCDATA)>
<!ATTLIST persona nacionalidad CDATA "Española">
```

Se define en él el atributo **nacionalidad** para el elemento **persona** como un atributo que contendrá texto de todo tipo, pero que por defecto toma el valor **Española** (nacionalidad por defecto en dicho archivo).

Entonces este archivo XML será válido:

XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE directorio SYSTEM "directorio.dtd">
<directorio>
        <persona nacionalidad="Francesa"> Kylian Mbappé</persona>
        <persona>Juan Martín</persona>
</directorio>
```

Entonces para **Kylian Mbappé** se ha indicado explícitamente la **nacionalidad**, pero no se ha usado el atributo para **Juan Martín**, por lo que tomará la nacionalidad española.

Valores fijos

Se puede utilizar el término **#FIXED** antes de indicar un valor fijo para un atributo. En ese caso en ningún documento XML se podrá modificar dicho atributo. Ejemplo:

DTD:

```
<!ELEMENT directorio (persona)+>
<!ELEMENT persona (#PCDATA)>
<!ATTLIST persona nacionalidad CDATA #FIXED "Española">
```

El atributo **nacionalidad** no podrá tomar ningún valor que no sea el valor **Española**, impidiendo tomar otra nacionalidad. En la práctica este tipo de atributos no se usa demasiado.



Codifica el **DTD** anterior y modifica el **XML** del apartado anterior para que sea válido. Después, comprueba que cualquier valor del atributo **nacionalidad**, que no sea **Española**, hace que el XML no sea válido.

Valores requeridos

En este caso se usa la palabra **#REQUIRED** indicando con ello que siempre hay que dar valor al atributo. Ejemplo:

DTD:

```
<!ATTLIST persona nacionalidad CDATA #REQUIRED>
```

Un documento XML que utilice el elemento **persona** deberá especificar obligatoriamente la **nacionalidad**.

Valor opcional

La palabra **#IMPLIED** especificada en el atributo indicaría que dicho atributo puede quedarse sin valor; es decir no posee valor por defecto, pero puede quedarse sin especificar (quedaría nulo por tanto).

DTD:

```
<!ATTLIST persona nacionalidad CDATA #IMPLIED>
```

En el ejemplo, **el atributo nacionalidad no es obligatorio especificarlo**, puede quedar sin valor. Nuevamente **en este caso no se puede especificar un valor por defecto (sería absurdo)**.

Tipos de atributo

CDATA

Los atributos de tipo **CDATA** permiten indicar como valor cualquier texto. A diferencia de los datos **PCDATA** de los elementos, los CDATA admiten cualquier carácter del tipo que sea.

ID

Sirve para generar **identificadores** a los elementos. **Un identificador es un valor único que tendrá cada elemento** y son muy usados en XML. El valor de un atributo de tipo ID cumple estas reglas:

- El valor tiene que cumplir las mismas reglas que para especificar nombres XML. Es decir: nada de espacios, no pueden comenzar con un número y sólo admite letras, números y el carácter de subrayado (_).
- No puede haber dos elementos con el mismo ID en un mismo documento XML
- En el DTD, para cada elemento sólo puede indicarse un atributo como ID. No puede haber dos atributos distintos en el mismo elemento que contengan ID.
- Los atributos ID sólo pueden indicar **#IMPLIED** o **#REQUIRED** en el apartado del valor por defecto.

IDREF

Los atributos IDREF contienen el valor de un ID de otro elemento. Es decir será una referencia a otro elemento. Las reglas de los IDREFs son:

- El valor de un IDREF debe cumplir las reglas para especificar nombres XML (es lógico ya que contienen valores de tipo ID)
- Debe existir un atributo ID en el documento XML cuyo valor coincida con el especificado en un IDREF (de otro modo se haría referencia a un elemento inexistente y esto no está permitido)

La idea es poder relacionar elementos a través de atributos de tipo ID e IDREF.

Analiza y prueba el siguiente código:

DTD: directorio.dtd

```
<!ELEMENT directorio (persona)+ >
<!ELEMENT persona (#PCDATA) >
<!ATTLIST persona id ID #REQUIRED
                  madre IDREF #IMPLIED
                  padre IDREF #IMPLIED>
```

XML:

```
<?xml version="1.0" encoding="UTF-8"?>

<!DOCTYPE directorio SYSTEM "directorio.dtd">
<directorio>
    <persona id="p1">Pedro</persona>
    <persona id="p2">Marisa</persona>
    <persona id="p3" madre="p2" padre="p1">Carmen</persona>
</directorio>
```

Carmen es la hija de Pedro y Marisa, según el código anterior, ya que los atributos **padre** y **madre** de tipo **IDREF** contienen los **ID** de **Pedro** y **Marisa**.



Prueba qué ocurre, al validar el XML, si cambias el valor de **madre** y **padre** con valores distintos a los que aparecen en el código. Por ejemplo: p4, p5, etc.

IDREFS

Igual que el anterior sólo que permite indicar varias referencias (que deben existir en el documento XML) a otros ID, **separadas por espacios**. De nuevo, analiza los siguientes códigos.

DTD: directorio.dtd

```
<!ELEMENT directorio (persona)+ >
<!ELEMENT persona (#PCDATA) >
<!ATTLIST persona id ID #REQUIRED padres IDREFS #IMPLIED >
```

XML:

```
<?xml version="1.0" encoding="UTF-8"?>

<!DOCTYPE directorio SYSTEM "directorio.dtd">
<directorio>
    <persona id="p1">Pedro</persona>
    <persona id="p2">Marisa</persona>
    <persona id="p3" padres="p1 p2">Carmen</persona>
</directorio>
```

ENUMERACIÓN

En este caso el **valor del atributo** debe de ser uno de una lista de valores posibles cada uno de los cuales se separa del siguiente mediante el símbolo **|**.

DTD:

```
<!ATTLIST persona sexo (Hombre | Mujer) #REQUIRED >
```

Las personas sólo podrán especificar como sexo **"Hombre"** o **"Mujer"** y nada más:

XML:

```
<persona sexo="Hombre">Javier Ruiz</persona>
```

Declaración de varios atributos en la misma etiqueta

Se usa muy habitualmente para indicar de forma cómoda todos los atributos de un determinado elemento:

Analiza los siguientes códigos para entender de una forma práctica este concepto. Después pruébalo.

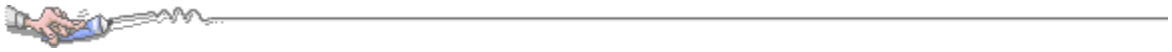
DTD: directorio.dtd

```
<!ELEMENT directorio (persona)+>
<!ELEMENT persona (#PCDATA)>
<!ATTLIST persona nacionalidad CDATA "Española" sexo (Hombre | Mujer)
#IMPLIED id ID #REQUIRED>
```

XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE directorio SYSTEM "directorio.dtd">
<directorio>
    <persona nacionalidad="Francesa" id="A1234">Vivian Maret</persona>
    <persona id="A789">Juan Martín</persona>
    <persona sexo="Hombre" id="A12">Rafael Díaz</persona>
</directorio>
```

6. Entidades



Las entidades son elementos XML que permiten indicar abreviaturas de texto (o referencias a elementos externos abreviadas) o utilizar caracteres que de otra forma serían inválidos en el documento.

Se explican sus posibilidades.

ENTIDADES YA EXISTENTES

En XML están definidas las siguientes entidades:

entidad	significado
<	El símbolo de menor (<)
>	El símbolo de mayor (>)
&	El ampersand: **&**
'	La comilla simple (')

Estas entidades no hay que declararlas en ningún DTD, todos los analizadores de XML estándar conocen estas entidades

Ejemplo:

XML:

```
<autor>Leopoldo Alas &apos;Clarín&apos;</autor>
```

El texto **PCDATA** del autor es **Leopoldo Alas 'Clarín'**. Si el archivo se abriera con Mozilla, esto es lo que se muestra:

```
−<directorio>
   <persona nacionalidad="Francesa" id="A1234">Vivian Maret</persona>
   <persona id="A789">Juan Martín</persona>
   <persona sexo="Hombre" id="A12">autor>Leopoldo Alas 'Clarín'</persona>
 </directorio>
```

ENTIDADES PARA REFERENCIAR CARACTERES ESPECIALES

La etiqueta inicial **<?xml** permite indicar (entre otras cosas) el juego de caracteres que utiliza un documento XML (normalmente **Unicode**, **UTF8**).

Si deseamos indicar un carácter especial que no está contenido en nuestro teclado, conociendo su código en el juego de caracteres que utiliza el documento, podemos especificarle con la sintaxis:

Utilizando números decimales:

```
&#número;
```

El número es el código del carácter en decimal. En hexadecimal se puede hacer con:

Utilizando números hexadecimales

```
&#xnúmero;
```

Observa estos códigos y cómo los visualiza el navegador Mozilla, por ejemplo.

Códigos:

```
<calle>Kantstra&#223;e, Berlín</calle>
<calle>Kantstra&#xDF;e, Berlín</calle>
```

Interpretación:

```
<calle>Kantstraße, Berlín</calle>
<calle>Kantstraße, Berlín</calle>
```

ENTIDADES GENERALES

Se usan como abreviaturas que aparecerán en el documento XML. La razón de su uso es facilitar la escritura de nombres repetitivos (nombres de la empresa, direcciones muy utilizadas,…). La sintaxis para declarar una entidad de este tipo es:

```
<!ENTITY nombre "texto">
```

Para usar en un documento XML la entidad declarada, se usa:

```
&nombre;
```

Ejemplo de declaración de una entidad:

DTD:

```
<!ENTITY mayor "Calle Mayor Principal" >
```

XML:

```
<dirección>&mayor; 18</dirección>
```

La dirección indicada es: Calle Mayor Principal 18.

Un uso muy interesante es usar entidades que hacen referencia a archivos externos (mediante su dirección URL), por ejemplo:

```
<!ENTITY direcciónCompleta SYSTEM "direccion.txt" >
```

Es la palabra **SYSTEM** la que indica que la entidad no es un texto sino que es el contenido de un archivo. El uso de la entidad **&direcciónCompleta;** en un documento XML provocará que en dicho documento se añada el contenido del archivo **dirección.txt** (en la posición exacta en la que esté colocada la referencia a la entidad).